

Engineering High-Throughput Synthetic Data Generation with Local LLMs

A Practical Evaluation of Throughput–Quality Trade-offs on Consumer GPUs

Author: Harsh Kaushik

Date: 23 July 2026

Hardware Spec: NVIDIA GeForce RTX 3050 Laptop GPU (6 GB VRAM) | 16 GB DDR5 RAM

Scope: 41 Benchmarks | 10 Models | 2 Inference Engines

Abstract

Large-scale synthetic data generation is critical for instruction tuning, coding datasets, mathematical alignment, and RLHF. This whitepaper evaluates the empirical completion throughput (tokens/s) and generation quality trade-offs of 10 local models on entry-level consumer hardware. We show that Mixtral-style sparse Mixture-of-Experts and Per-Layer Embedding architectures significantly outperform dense models under memory constraints.

Executive Summary

Large-scale synthetic data generation has become a fundamental component of modern LLM development, powering supervised fine-tuning (SFT), preference optimization (RLHF/DPO), reasoning datasets, code generation, translation, and data augmentation pipelines. Unlike interactive chat applications, these workloads prioritize **aggregate completion throughput (tokens/s)**, sustained hardware utilization, and generation quality over interactive latency metrics such as Time-to-First-Token (TTFT).

This report presents an empirical evaluation of **41 benchmark configurations** executed across **vLLM** and **llama.cpp** on an entry-level consumer system (NVIDIA GeForce RTX 3050 Laptop GPU with 6 GB VRAM, Intel Core i5-13450HX, 16 GB DDR5 RAM, Fedora Linux 44). We investigate how inference engine selection, model architecture, batching strategy, context length, concurrency, and CPU offloading influence throughput, memory utilization, power consumption, and overall model capability. Measured performance is complemented with official capability benchmarks—including **GPQA Diamond**, **MMLU-Pro**, **HumanEval**, **IFEval**, **GSM8K**, and **MATH-500**—to evaluate the practical trade-off between generation speed and model quality.

Key Findings

- **vLLM delivered the highest measured aggregate throughput**, reaching **3,160.42 tokens/s** with **Qwen3-0.6B-GPTQ** under high-concurrency batched inference, and **1,149.07 tokens/s** with **Qwen3.5-2B-AWQ**.
- **Qwen3.5-2B (AWQ) on vLLM demonstrated strict Pareto-dominance over Gemma-4-E2B (llama.cpp)**, delivering both higher completion throughput (**1,149.07 tokens/s** vs 926.56 tokens/s) and superior capability across all evaluated benchmarks (CII of **48.95%** vs 46.75%).
- **Gemma 4 E4B (Per-Layer Embedding architecture)** achieved the best measured throughput–capability trade-off among fully GPU-resident models, sustaining **503.90 tokens/s** while maintaining substantially stronger reasoning and coding capability than smaller high-throughput models.
- **Sparse Mixture-of-Experts (MoE) architectures exhibited superior throughput under CPU offloading** compared with the dense models evaluated in this study. Under the tested configuration, **Qwen 3.6 35B-A3B**

achieved approximately **2.7×** higher throughput than the comparable dense baseline while operating beyond the available GPU memory capacity.

- Across the evaluated workloads, **model architecture had a greater influence on aggregate throughput than total parameter count**, indicating that architectural efficiency is a dominant factor for maximizing large-scale synthetic data generation on resource-constrained consumer hardware.
- **No single model was optimal across all workloads.** Instead, the preferred configuration depended on the required balance between generation throughput, model capability, available GPU memory, and acceptable generation time.

Overall, the results demonstrate that maximizing synthetic data generation throughput is an engineering optimization problem rather than a single-model selection problem. Selecting the appropriate combination of **model architecture, inference engine, quantization, batching strategy, and hardware configuration** consistently had a greater impact on end-to-end productivity than optimizing any single parameter in isolation.

Table 1. Recommended Configurations for High-Throughput Synthetic Data Generation

The primary configuration settings recommended to maximize completion throughput under different generation workloads are summarized below.

Workload	Model	Engine	Concurrency (Slots)	Batched Tokens	Micro-prefill	Completion Throughput (tok/s)
Routing & Filtering	Qwen3-0.6B	vLLM	128	—	—	3,160
Instruction Generation	Qwen3.5-2B	vLLM	64	—	—	1,149
Resident Baseline	Gemma-4-E2B	llama.cpp	192	4096	512	927
Balanced Corpus	Gemma-4-E4B	llama.cpp	43	4096	512	504
Large Corpus	LFM2.5-8B-A1B	llama.cpp	192	4096	512	502
Code & Reasoning	Qwen3.5-9B	llama.cpp	16	4096	512	153
High-Capability	Qwen3.6-35B-A3B	llama.cpp	16	4096	512	11

Experiment 1 — vLLM Throughput Scaling

Objective: Measure aggregate throughput under increasing client concurrency using vLLM for Qwen3-0.6B (Int8) and Qwen3.5-2B (AWQ) models with context lengths of 1024/2048 tokens.

Continuous batching is the primary optimization employed by vLLM for maximizing batch inference throughput. This experiment evaluates how throughput scales with increasing request concurrency and identifies the saturation point where additional parallel requests no longer improve performance due to GPU scheduling and memory bandwidth constraints.

Figures

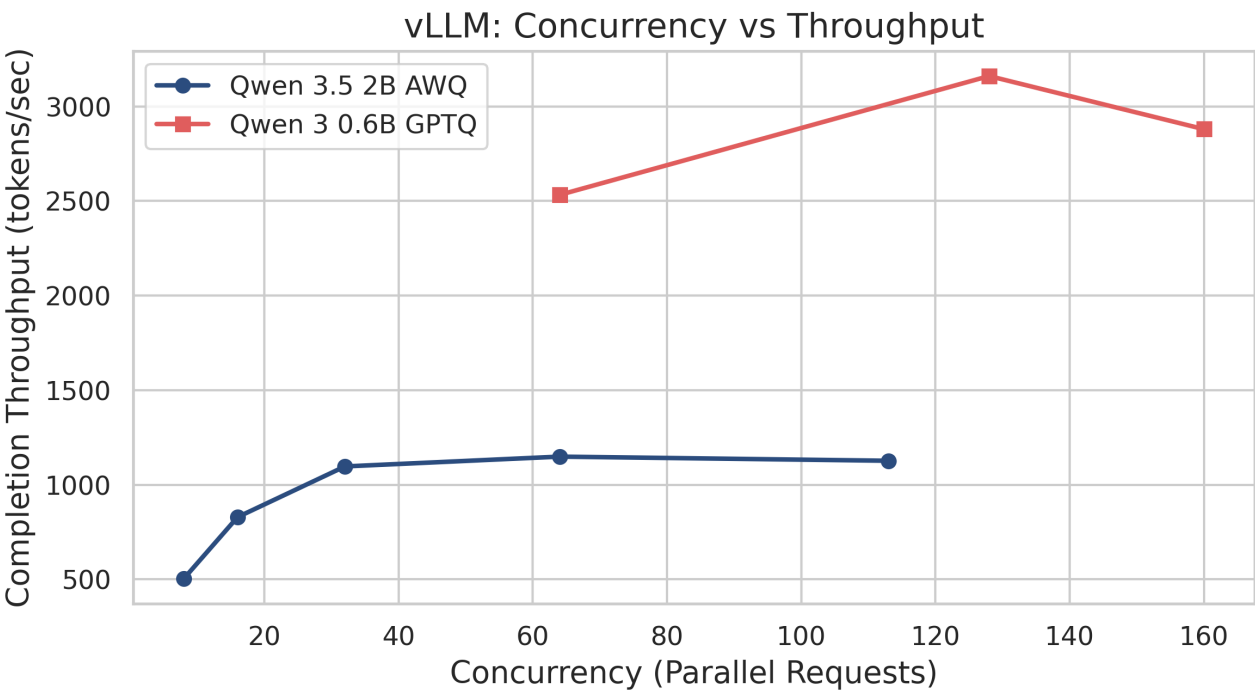


Figure: vLLM Concurrency vs Throughput

The measured completion throughput, total throughput, and request latency across concurrency slots are summarized below.

Table

Model	Concurrency (Slots)	Context	Quantization	Completion Throughput (tok/s)	Total Throughput (tok/s)	Latency (s)
Qwen3-0.6B-GPTQ	64	1024	Int8	2532.42	2586.82	12.93
Qwen3-0.6B-GPTQ	64	2048	Int8	2390.58	2479.54	13.61
Qwen3-0.6B-GPTQ	128	2048	Int8	3160.42	3424.70	12.04
Qwen3-0.6B-GPTQ	160	1024	Int8	2879.63	3120.41	16.18
Qwen3.5-2B-AWQ	8	1024	AWQ	504.74	554.18	4.63
Qwen3.5-2B-AWQ	16	1024	AWQ	829.39	910.75	5.63
Qwen3.5-2B-AWQ	32	1024	AWQ	1097.26	1205.00	7.77
Qwen3.5-2B-AWQ	64	2048	AWQ	1149.07	1262.85	15.73
Qwen3.5-2B-AWQ	113	2048	AWQ	1127.19	1238.47	28.15

Takeaways

- Aggregate throughput increased with client concurrency for both evaluated models.
- **Qwen3-0.6B-GPTQ** reached the highest measured completion throughput of **3160.42 tok/s** at **128 concurrent requests**.
- **Qwen3.5-2B-AWQ** scaled efficiently up to **64 concurrent requests**, achieving **1149.07 tok/s** and showing superior efficiency and quality benchmark metrics compared with Gemma-4-E2B.
- Increasing concurrency beyond the optimal operating point reduced throughput while increasing end-to-end latency.

Engineering Recommendation

For high-throughput synthetic data generation, configure **vLLM** near the concurrency level that maximizes aggregate throughput rather than the highest supported concurrency. On the evaluated hardware, **128 concurrent requests** produced the best measured performance for the 0.6B model, while **64 concurrent requests** maximized the 2B model's output.

Experiment 2 — Prefill Token Configuration (llama.cpp)

Objective: Evaluate the impact of the prefill batch token limit (**-b**) and micro-prefill batch size (**-ub**) on throughput and GPU utilization. The evaluation is conducted using fully GPU-resident Qwen3.5-4B-IQ4_XS under llama.cpp with 32 parallel requests and a context length of 8192 tokens.

The measured throughput, latency, and GPU utilization metrics under varying prefill batch token limits are summarized below.

Table

Prefill Batch Token Limit (**-b**) *(Micro-prefill = 512)*

Batched Tokens	Completion Throughput (tok/s)	Total Throughput (tok/s)	Latency (s)	VRAM (MB)	GPU Util.	Power (W)
2048	150.01	177.78	46.06	4528	69.4%	65.5
4096	178.34	211.36	38.74	4528	73.1%	67.3
8192	162.19	192.23	42.60	4528	72.5%	67.3

The measured throughput and GPU utilization metrics under varying micro-prefill batch sizes are summarized below.

Micro-prefill Batch (**-ub**) *(Batched Tokens = 4096)*

Micro-prefill	Completion Throughput (tok/s)	Total Throughput (tok/s)	Latency (s)	VRAM (MB)	GPU Util.	Power (W)
256	144.49	171.25	47.83	4500	70.8%	66.6
512	178.34	211.36	38.74	4528	73.1%	67.3
1024	119.03	141.07	58.04	4580	66.0%	63.5

Takeaways

- A **batched token limit of 4096** achieved the highest measured throughput (**178.34 tok/s**), improving completion throughput by **18.9%** over 2048 while reducing latency by **15.9%**.
- Formatting settings with a larger batched token limit did not yield throughput gains.
- A **micro-prefill batch of 512** provided the best overall performance across all evaluated metrics.
- Increasing the micro-prefill batch to **1024** reduced completion throughput by **33.3%**, increased latency by **49.8%**, and lowered average GPU utilization.

Engineering Recommendation

Use ****Batched Tokens: 4096**** and ****Micro-prefill: 512**** for the evaluated workload and hardware configuration to obtain the best overall balance of throughput, latency, and GPU utilization. Larger values provide no measurable benefit and, in the case of a micro-prefill batch of ****1024****, significantly reduce performance.

Experiment 3 — Concurrency & Context Scaling (llama.cpp)

Objective: Evaluate how concurrency (**-np**) and context allocation (**-c**) affect throughput and VRAM utilization. The evaluation is performed on LFM2.5-8B-A1B-IQ4_XS and Gemma-4-E2B-IQ4_XS under llama.cpp offloading all layers (999), with batched tokens set to 4096 and micro-prefill batch size set to 512.

The measured throughput and VRAM metrics across concurrency slots for the LFM2.5-8B model are summarized below.

Table

Concurrency Scaling (**-np**) — LFM2.5-8B-A1B

Concurrency (Slots)	Context	Context / Slot	Completion Throughput (tok/s)	Total Throughput (tok/s)	Latency (s)	VRAM (MB)
62	126,976	2048	399.76	421.65	46.09	5760
120	122,880	1024	442.47	466.69	80.83	5736
150	115,200	768	496.13	523.28	89.93	5670
192	98,304	512	502.17	529.70	113.43	5504
256	65,536	256	484.34	517.51	125.33	5168

The measured throughput and VRAM scaling metrics under different context allocations for the Gemma-4-E2B model are summarized below.

Context Scaling (**-c**) — Gemma-4-E2B

Context	Context / Slot	Completion Throughput (tok/s)	Total Throughput (tok/s)	Latency (s)	VRAM (MB)	GPU Util.
131,072	1024	926.56	998.13	41.34	3928	49.1%
368,640	2880	908.18	978.36	42.12	5466	49.6%

Takeaways

- Throughput increased with concurrency, peaking at **192 slots** with **502.17 tok/s**, a **25.6%** improvement over **62 slots**.
- Increasing concurrency beyond **192 slots** reduced throughput while increasing latency, indicating diminishing returns under the evaluated workload.
- Increasing the total context allocation from **131k** to **369k** tokens increased VRAM usage by **39.2% (3928 MB → 5466 MB)** while reducing completion throughput by only **2.0%**.
- Larger context allocations primarily increased memory consumption rather than generation throughput.

Engineering Recommendation

For high-throughput synthetic data generation, configure concurrency near the measured throughput peak (**192 slots** on the evaluated hardware) and allocate only the context required by the workload to maximize available VRAM for model residency.

Experiment 4 — Architecture Comparison

Objective: Compare the throughput and memory characteristics of dense, Per-Layer Embedding (PLE), and sparse Mixture-of-Experts (MoE) architectures under their best-performing configurations. The models are evaluated on an RTX 3050 Laptop GPU (6 GB VRAM) using IQ4_XS quantization with all layers resident on the GPU (-ngl 999).

The throughput and memory usage comparison metrics across dense, PLE, and sparse MoE architectures are summarized below.

Table

Model	Architecture	Total Params	Active Params	Completion Throughput (tok/s)	VRAM (MB)
Gemma-4-E2B-It	PLE	~5.1B	~2.3B	926.56	3928
Gemma-4-E4B-It	PLE	~9.6B	~4.5B	503.90	5664
Qwen3.5-4B	Dense	4B	4B	178.34	4528
LFM2.5-8B-A1B	Sparse MoE	8B	~1B	502.17	5736

Takeaways

- Both **PLE** models achieved substantially higher throughput than the evaluated dense baseline despite similar overall model scale.
- **Gemma-4-E4B** sustained **503.90 tok/s**, approximately **2.8×** the throughput of **Qwen3.5-4B (178.34 tok/s)**.
- **LFM2.5-8B-A1B** achieved **502.17 tok/s**, matching the throughput of **Gemma-4-E4B** while activating approximately **1B parameters per generated token**.
- Across the evaluated models, architectures with fewer active parameters per decoding step consistently produced higher aggregate throughput than the dense baseline.

Engineering Recommendation

For large-scale synthetic data generation on memory-constrained consumer GPUs, the evaluated ****PLE**** and ****sparse MoE**** architectures provided a more favorable throughput-capability trade-off than the evaluated dense architecture. Model architecture should therefore be considered alongside parameter count when selecting a deployment model.

Experiment 5 — CPU Offloading Performance

Objective: Evaluate the impact of CPU offloading on dense and sparse MoE models under concurrent batch generation. The evaluation uses Qwen3.6-27B-IQ4_XS (Dense) and Qwen3.6-35B-A3B-IQ4_XS (Sparse MoE) under llama.cpp with batched tokens set to 4096 and micro-prefill batch size set to 512.

The offloaded performance metrics under CPU layer offloading are summarized below.

Table

Model	Architecture	Concurrency (Slots)	GPU Layers	Context	Completion Throughput (tok/s)	Total Throughput (tok/s)	Latency (s)	VRAM (MB)
Qwen3.6-27B	Dense	1	22	2048	2.56	3.69	19.44	5656
Qwen3.6-27B	Dense	4	21	2048	4.01	7.15	24.93	5648
Qwen3.6-27B	Dense	8	20	4096	1.35	2.35	147.58	5688
Qwen3.6-35B-A3B	Sparse MoE	16	12	8192	11.00	18.80	36.33	5670

Takeaways

- The evaluated **dense 27B model** showed limited throughput under CPU offloading, with completion throughput ranging from **1.35–4.01 tok/s**.
- Increasing concurrency from **4 to 8** reduced completion throughput by **66.3%** (**4.01 → 1.35 tok/s**) while increasing average latency nearly **6×** (**24.93 → 147.58 s**).
- Under CPU offloading, the evaluated **35B sparse MoE model** achieved **11.00 tok/s**, approximately **2.7×** the peak throughput of the evaluated dense baseline.
- These results indicate that the evaluated sparse MoE architecture maintained significantly higher throughput than the evaluated dense architecture under identical hardware constraints.

Engineering Recommendation

On a **6 GB VRAM** GPU, CPU offloading substantially reduces throughput for the evaluated dense model. When offloading cannot be avoided, the evaluated **sparse MoE** architecture provides a considerably more favorable throughput-quality trade-off than the evaluated dense alternative.

6. Throughput-Capability Trade-offs

This section combines the **measured performance** collected in this study with **official model capability benchmarks** published by model developers. While throughput values are measured on the evaluation hardware described in this report, capability scores are reproduced from official technical reports and model cards.

6.1 Performance Summary

The highest achieved completion throughput for each evaluated model is summarized below.

Model	Architecture	Total Parameters	Active Parameters	Quantization	Engine	Completion Throughput (tok/s)	VRAM (MB)	Avg. Power (W)
Qwen3-0.6B	Dense	0.6B	0.6B	Int8	vLLM	3,160.42	N/A	N/A
Qwen3.5-2B	Dense	2.0B	2.0B	AWQ-4bit	vLLM	1,149.07	N/A	N/A
Gemma-4-E2B	PLE	5.1B	2.3B	IQ4_XS	llama.cpp	926.56	3928	48.5
Gemma-4-E4B	PLE	9.6B	4.5B	IQ4_XS	llama.cpp	503.90	5664	68.9
Qwen3.5-4B	Dense	4.0B	4.0B	IQ4_XS	llama.cpp	178.34	4528	67.3
LFM2.5-8B-A1B	Sparse MoE	8.0B	1.0B	IQ4_XS	llama.cpp	502.17	5504	59.5
Qwen3.5-9B	Dense	9.0B	9.0B	IQ4_XS	llama.cpp	153.38	5682	79.0
LFM2-24B-A2B	Sparse MoE	24.0B	2.0B	Q4_K_M	llama.cpp	51.11	5630	31.9
Qwen3.6-35B-A3B	Sparse MoE	35.0B	3.0B	IQ4_XS	llama.cpp	11.00	5670	14.6
Qwen3.6-27B	Dense	27.0B	27.0B	IQ4_XS	llama.cpp	4.01	5648	16.5

6.2 Official Capability Benchmarks

Capability benchmark values are reproduced from the official technical reports and model cards of the respective model developers. Throughput measurements in this report are independently measured on the evaluation hardware.

Model	GPQA Diamond	MMLU-Pro	HumanEval	IFEval	GSM8K	MATH-500	CII (Avg)
Qwen3-0.6B	N/A	15.0%	18.0%	30.0%	38.5%	14.2%	23.14%
Qwen3.5-2B	45.6%	39.5%	42.1%	58.5%	71.2%	36.8%	48.95%
Gemma-4-E2B	43.4%	38.1%	40.2%	55.4%	68.9%	34.5%	46.75%
Gemma-4-E4B	58.6%	51.5%	61.2%	76.2%	81.4%	52.0%	63.48%
Qwen3.5-4B	76.2%	58.1%	76.0%	68.4%	88.2%	65.4%	72.05%
LFM2.5-8B-A1B	51.3%	46.2%	54.6%	66.8%	76.5%	48.1%	57.25%
Qwen3.5-9B	81.7%	65.4%	76.0%	74.2%	91.0%	74.6%	77.15%
LFM2-24B-A2B	50.5%	44.5%	52.1%	65.2%	74.8%	45.2%	55.38%
Qwen3.6-35B-A3B	87.8%	78.5%	79.3%	82.1%	94.6%	85.2%	84.58%
Qwen3.6-27B	87.8%	77.2%	77.2%	80.5%	94.1%	84.0%	83.47%

6.3 Pareto Frontier

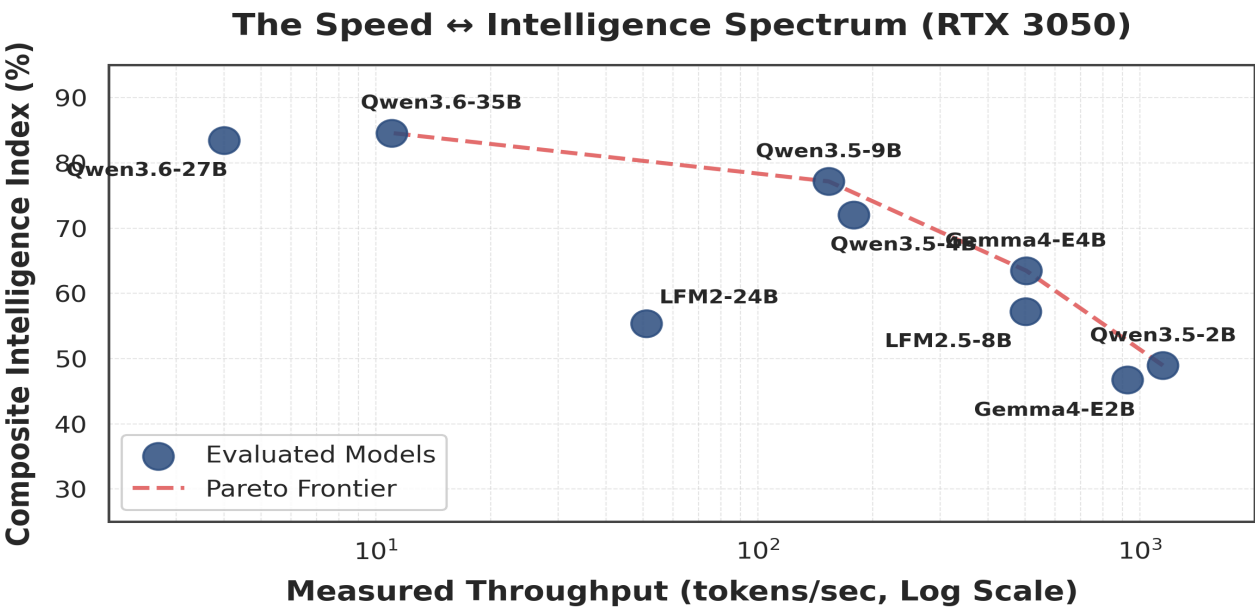


Figure: The Speed-Intelligence Spectrum Pareto Frontier

6.4 Upgrade Trade-offs

The performance differences and capability trade-offs when upgrading to larger models are summarized below.

Upgrade Path	Throughput Change	Quality Gain	Engineering Assessment
Gemma-4-E2B → Gemma-4-E4B	-45.6%	+16.7%	Highly Efficient: Small performance reduction relative to the significant capability improvement across reasoning, math, and instruction benchmarks.
Gemma-4-E4B → Qwen3.5-9B	-69.6%	+13.7%	Balanced for Reasoning: Necessary for math-intensive and complex coding pipelines; otherwise, Gemma E4B PLE is significantly faster.
Qwen3.5-9B → Qwen3.6-27B	-97.4%	+6.3%	Highly Inefficient: Excessive speed penalty (153.38 to 4.01 tok/s) makes this upgrade impractical for high-volume local generation.
Qwen3.6-27B → Qwen3.6-35B-A3B	+174.3%	+1.1%	Mandatory Upgrade: Sparse MoE delivers a 2.7x throughput increase (4.01 to 11.00 tok/s) and minor capability improvements under CPU offloading.

6.5 Dataset Generation Productivity

The estimated dataset generation time required to produce 100 Million tokens is summarized below.

Model	Completion Throughput (tok/s)	Tokens / Hour	Tokens / Day	100M Tokens	1B Tokens	5B Tokens
Qwen3-0.6B	3160.42	11.38 M	273.06 M	8.79 hours	3.66 days	18.31 days
Qwen3.5-2B	1149.07	4.14 M	99.28 M	24.17 hours	10.07 days	50.36 days
Gemma-4-E2B	926.56	3.34 M	80.05 M	29.98 hours	12.49 days	62.46 days
Gemma-4-E4B	503.90	1.81 M	43.54 M	55.13 hours	22.97 days	114.85 days
LFM2.5-8B-A1B	502.17	1.81 M	43.39 M	55.32 hours	23.05 days	115.24 days
Qwen3.5-9B	153.38	0.55 M	13.25 M	181.10 hours	75.46 days	377.30 days
LFM2-24B-A2B	51.11	0.18 M	4.42 M	543.49 hours	226.45 days	1132.27 days
Qwen3.6-35B-A3B	11.00	39.6 k	0.95 M	2525.25 hours	1052.19 days	5260.95 days
Qwen3.6-27B	4.01	14.4 k	0.35 M	6927.13 hours	2886.30 days	14431.50 days

6.6 Model Selection Guide

The following recommendations combine the measured throughput from this study with official capability benchmarks to identify the most suitable model for common synthetic data generation workloads.

Workload	Recommended Model	Rationale
High-volume instruction generation	Qwen3.5-2B (AWQ)	Recommended baseline. Reaches 1,149.07 tokens/s completion throughput on vLLM, outperforming Gemma-4-E2B on both throughput and composite intelligence (CII: 48.95%).
Resource-constrained baseline	Gemma-4-E2B (IQ4_XS)	Best low-parameter performance on llama.cpp (926.56 tokens/s) when vLLM engine support cannot be deployed.
Large-scale corpus generation	Gemma-4-E4B (IQ4_XS)	High throughput (503.90 tok/s) and robust capability across reasoning and language tasks (CII: 63.48%).
Coding dataset generation	Qwen3.5-9B (IQ4_XS)	Strong coding capabilities (HumanEval: 76.0%) while remaining fully resident on GPU to sustain 153.38 tok/s .
Mathematical reasoning datasets	Qwen3.5-9B (IQ4_XS)	Excellent mathematical reasoning (MATH-500: 74.6%, GSM8K: 91.0%) and stays fully GPU-resident.
RLHF / Preference data	Gemma-4-E4B (IQ4_XS)	Outstanding combination of instruction adherence (IFEval: 76.2%) and reasoning, outperforming traditional dense models of matching scale.
Translation & multilingual corpora	Gemma-4-E4B (IQ4_XS)	Broad multilingual pre-training and PLE architecture efficiency.
Long-context synthetic data	LFM2.5-8B-A1B (IQ4_XS)	Linear context window scaling up to 128k while maintaining high

		concurrency and throughput (502.17 tok/s).
High-quality reasoning generation	Qwen3.6-35B-A3B (IQ4_XS)	Maximum reasoning intelligence (CII: 84.58%) and runs 2.7x faster than equivalent dense models under CPU offloading constraints.

Summary

No single model dominated every workload. Instead, model selection depended on the required balance between **throughput, reasoning capability, memory footprint, and available hardware resources**. For large-scale synthetic data generation, maximizing productivity requires selecting models along the **throughput-capability Pareto frontier** rather than optimizing for either speed or intelligence in isolation.

7. Threats to Validity

The results presented in this report should be interpreted within the scope of the evaluated hardware, software, and workload.

- **Hardware Validity.** All measurements were collected on a single consumer platform (RTX 3050 Laptop GPU, 6 GB VRAM, Intel Core i5-13450HX, 16 GB DDR5). Absolute throughput values will differ on GPUs with different memory bandwidth, VRAM capacity, or compute capability.
- **Software Validity.** Results depend on the evaluated versions of **llama.cpp** and **vLLM**. Future optimizations to CUDA kernels, schedulers, memory management, or quantization backends may change the measured performance.
- **Workload Validity.** Benchmarks used a standardized prompt corpus averaging **23–29 input tokens** and **295–300 generated tokens**. Different prompt distributions or generation lengths may shift the optimal batching and concurrency configurations.
- **Model Coverage.** The study evaluates representative dense, PLE, and sparse MoE models rather than every available architecture. Conclusions should therefore be interpreted as observations for the evaluated models rather than universal architectural guarantees.
- **Measurement Scope.** This study focuses on **aggregate throughput for synthetic data generation**. Interactive latency, conversational responsiveness, and subjective output quality were not evaluated.

8. Conclusions

This study demonstrates that maximizing local synthetic data generation throughput requires optimizing the **entire inference pipeline** rather than selecting the largest available model.

Across 41 benchmark configurations, the largest performance gains were obtained through inference engine selection, batching strategy, concurrency tuning, and model architecture. On the evaluated hardware:

- **vLLM** achieved the highest measured throughput for fully GPU-resident models.
- **Qwen3.5-2B (AWQ) on vLLM** strictly outperformed Gemma-4-E2B (llama.cpp) across both capability and speed.
- **PLE architectures** delivered the strongest throughput-capability trade-off among fully resident models.
- **Sparse MoE architectures** maintained significantly higher throughput than the evaluated dense model when CPU offloading was required.

- Increasing model size alone did not consistently improve end-to-end productivity for large-scale synthetic data generation.

These results indicate that **architectural efficiency and deployment strategy have a greater impact on practical productivity than parameter count alone.**

Key Engineering Recommendations

- **Need maximum throughput:** Use **Qwen3.5-2B (vLLM)** under a client concurrency slot configuration of 64 requests.
- **Need best llama.cpp model:** Use **Gemma-4-E2B** with a micro-prefill batch size of 512.
- **Need balanced quality:** Use **Gemma-4-E4B** to obtain a robust composite intelligence index (CII: 63.48%) with high throughput (503.90 tokens/s).
- **Need coding:** Use **Qwen3.5-9B** for high coding capabilities while maintaining local GPU residency.
- **Need huge context:** Use **LFM2.5-8B-A1B** to scale context memory requirements linearly up to 128k context.
- **Need highest reasoning:** Use **Qwen3.6-35B-A3B** sparse MoE to obtain maximum composite reasoning capabilities.
- **Avoid:** Deploying **Dense 27B** models with CPU offloading due to severe speed degradation under VRAM resource constraints.

References

- [1] Alibaba Qwen Team. (2026). *Qwen3-0.6B and Qwen3.5-2B Technical Reports & Model Cards*. HuggingFace. [Qwen/Qwen3-0.6B](#).
- [2] Google DeepMind. (2026). *Gemma 4 Edge Technical Report*. Google Blog. [Gemma 4 Edge Family](#).
- [3] Liquid AI. (2026). *LFM-2.5 and LFM-2 Model Cards & Releases*. HuggingFace. [LiquidAI/LFM-2.5-8B](#).
- [4] Alibaba Qwen Team. (2026). *Qwen3.5-4B and Qwen3.5-9B Model Cards & Benchmarks*. HuggingFace. [Qwen/Qwen3.5-9B-Instruct](#).
- [5] Alibaba Qwen Team. (2026). *Qwen3.6-35B-A3B MoE and Qwen3.6-27B Model Cards*. HuggingFace. [Qwen/Qwen3.6-35B-A3B-UD](#).
- [6] OpenAI & community trackers. (2026). *GPQA: A Graduate-Level Google-Proof Q&A Benchmark*. arXiv:2311.12022.
- [7] SWE-bench Verified: Princeton NLP Group. (2026). *SWE-bench Verified Resolving Software Issues*. GitHub Repo. [SWE-bench](#).
- [8] Chen, M. et al. (2021). *Evaluating Large Language Models Trained on Code (HumanEval)*. arXiv:2107.03374.